Informe Laboratorio 4

Sección 3

Edicson Solar Salinas
e-mail: edicson.solar@mail.udp.cl

26 de Octubre a 1 de Noviembre de 2026

Índice

1. Descripción **2**

2. Actividades **2**
2.1. Configuración y análisis de SSH . . . 2
2.1.1. Implementación y configuración del servidor SSH . . . 2
2.1.2. Análisis de paquetes SSH en Wireshark . . . 2
2.2. Análisis de captura de tráfico antigua . . . 3
2.2.1. Análisis de los mecanismos criptográficos . . . 3
2.2.2. Recuperación del PIN y derivación de la clave AES . . . 3
2.2.3. Descifrado de las transacciones y análisis final . . . 3

3. Desarrollo de actividades **4**
3.1. Actividad 1 . . . 4
3.1.1. Dockerfile . . . 4
3.1.2. Configuración del servidor . . . 6
3.1.3. Explicación de los algoritmos utilizados . . . 7
3.1.4. Conexión cliente-servidor . . . 7
3.1.5. Análisis de tráfico . . . 8
3.2. Actividad 2 . . . 10
3.2.1. Identificación del paquete RSA_PIN . . . 10
3.2.2. Identificación de los paquetes AESGCM . . . 11
3.2.3. Recuperación del PIN y derivación de la clave AES . . . 12
3.2.4. Descifrar los mensajes . . . 12
3.2.5. RSA frente a criptografía post-cuántica . . . 16

Conclusiones y comentarios **16**

4. Bibliografia **17**

1. Descripción

Usted trabaja en una empresa especializada en la transferencia segura de información, archivos y contenido digital. Antes de incorporarse al desarrollo y operación de los sistemas reales de la organización, se le solicita realizar una demostración técnica que permita evaluar sus conocimientos en redes, protocolos de comunicación y criptografía.

Para ello, deberá implementar y configurar un servidor SSH, definiendo los mecanismos criptográficos utilizados para proteger la comunicación. Posteriormente, deberá establecer una conexión con el servidor y capturar el tráfico de red utilizando Wireshark.

A partir del análisis de los paquetes capturados, deberá identificar qué información puede ser observada por un tercero que tuviera acceso al tráfico de red y determinar qué elementos de la comunicación están protegidos mediante cifrado y cuáles pueden ser visibles, con el objetivo de evaluar el nivel de seguridad proporcionado por el protocolo SSH.

Finalmente, se le proporcionará una captura de tráfico de red obtenida mediante Wireshark. En ella se encuentran diversos mensajes correspondientes a transacciones que han sido cifrados mediante RSA.

El objetivo de esta actividad es analizar la captura de tráfico, identificar los paquetes que contienen la información cifrada y desarrollar un programa en Python que permita recuperar el contenido original.

2. Actividades

2.1. Configuración y análisis de SSH

2.1.1. Implementación y configuración del servidor SSH

Generar un contenedor Docker que permita ejecutar un servidor SSH accesible desde el equipo cliente (su computador). Posteriormente, configure el servidor mediante un archivo `sshd_config`, en el cual deberá especificar los algoritmos criptográficos utilizados durante la comunicación SSH: `curve25519-sha256`, `aes256-ctr`, `hmac-sha2-256` y `ssh-ed25519`. Por último, documente la función que cumple cada algoritmo dentro del proceso de establecimiento y comunicación de una conexión SSH.

2.1.2. Análisis de paquetes SSH en Wireshark

Una vez completada la configuración del servidor SSH, deberá analizar el tráfico de red generado durante el establecimiento de la conexión utilizando Wireshark.

Para ello, antes de realizar la conexión SSH, inicie la captura de tráfico en la interfaz de red correspondiente. Posteriormente, establezca una conexión con el servidor utilizando su máquina como cliente y analice los paquetes intercambiados durante el proceso de conexión.

A partir del análisis realizado, responda la siguiente pregunta:

¿Qué información es posible identificar a partir de los paquetes capturados? Mencione y explique al menos tres elementos que puedan observarse en el tráfico de la conexión SSH.

2.2. Análisis de captura de tráfico antigua

2.2.1. Análisis de los mecanismos criptográficos

Se le solicita analizar una captura de tráfico de red antigua, en formato .pcap, que contiene transacciones financieras cifradas mediante una combinación de criptografía asimétrica y simétrica. Para realizar la actividad, se le proporcionarán la captura y las claves pública y privada RSA correspondientes. Deberá identificar los paquetes relevantes y analizar el mecanismo de cifrado utilizado, considerando que RSA se emplea para cifrar un PIN de cuatro dígitos mediante OAEP con SHA-256, mientras que AES-GCM se utiliza para proteger el contenido de las transacciones financieras.

2.2.2. Recuperación del PIN y derivación de la clave AES

El primer paquete relevante de la captura contiene la etiqueta RSA_PIN, seguida de una salt de 16 bytes y del PIN cifrado mediante RSA. Deberá identificar y extraer estos campos, utilizar la clave privada RSA proporcionada para recuperar el PIN y, posteriormente, derivar la clave AES de 256 bits mediante Scrypt. Para ello, deberá utilizar el PIN recuperado, la salt extraída de la captura y los siguientes parámetros: (n=2**14), (r=8) y (p=1). Explique la función de la salt y por qué es necesario utilizar los mismos parámetros para obtener la clave AES original.

2.2.3. Descifrado de las transacciones y análisis final

Una vez obtenida la clave AES, deberá identificar todos los paquetes que contienen transacciones cifradas mediante AES-GCM. Para cada paquete, deberá extraer el nonce de 12 bytes y utilizarlo junto con la clave derivada y el texto cifrado para recuperar los mensajes originales, verificando su autenticidad. Implemente el procedimiento completo en un script de Python y presente las transacciones recuperadas en su formato original, incluyendo el identificador, el remitente, el destinatario y el monto. Documente el procedimiento realizado e incluya evidencias de los resultados obtenidos.



Figura 1: Ejemplo del resultado

Finalmente, responda la siguiente pregunta, justificando su respuesta: ¿Sería posible descifrar los mensajes capturados utilizando las claves pública y privada RSA en cifrados PQC (Criptografía post-cuántica)?

3. Desarrollo de actividades

3.1. Actividad 1

3.1.1. Dockerfile

Usé una imagen base `debian:bookworm-slim` e instalé `openssh-server`. Dentro de la imagen creé el usuario `lab4`, copié su clave pública y la configuración del servidor, y generé una clave de host Ed25519. La autenticación del usuario quedó limitada a clave pública.

Antes de terminar la construcción ejecuté `sshd -t`. De esta forma, una configuración inválida hace fallar el build en vez de aparecer recién al iniciar el contenedor. El proceso principal se ejecuta en primer plano con `sshd -D -e`.

```
lavapatos@debian:~/Documents/uCosasLocal/2026-2/Cripto/labs/lab4$ touch evidencias/configuracion/Dockerfile
lavapatos@debian:~/Documents/uCosasLocal/2026-2/Cripto/labs/lab4$ sudo nano evidencias/configuracion/Dockerfile
lavapatos@debian:~/Documents/uCosasLocal/2026-2/Cripto/labs/lab4$ cat evidencias/configuracion/Dockerfile
FROM debian:bookworm-slim

RUN apt-get update && apt-get install -y openssh-server && rm -rf /var/lib/apt/lists/*

RUN useradd -m -s /bin/bash lab4 && passwd -d lab4
RUN mkdir -p /home/lab4/.ssh && chown lab4:lab4 /home/lab4/.ssh && chmod 700 /home/lab4/.ssh

COPY --chown=lab4:lab4 --chmod=600 authorized_keys /home/lab4/.ssh/authorized_keys
COPY sshd_config /etc/ssh/sshd_config

RUN rm -f /etc/ssh/ssh_host_* && ssh-keygen -q -t ed25519 -N '' -f /etc/ssh/ssh_host_ed25519_key
RUN mkdir -p /run/sshd && /usr/sbin/sshd -t -f /etc/ssh/sshd_config

EXPOSE 22

CMD ["/usr/sbin/sshd", "-D", "-e", "-f", "/etc/ssh/sshd_config"]
lavapatos@debian:~/Documents/uCosasLocal/2026-2/Cripto/labs/lab4$
```

Figura 2: Dockerfile utilizado para construir el servidor SSH.

Construí la imagen con el nombre `lab4-ssh:2026`. El proceso terminó correctamente y creó la imagen local, como se observa en la Figura 3.

Después inicié el contenedor `lab4-ssh-server`. El puerto 22 del contenedor se publicó solamente como `127.0.0.1:22222`, por lo que el servidor quedó accesible desde el mismo equipo y no desde otras interfaces de red.

El registro del contenedor confirmó que `sshd` quedó escuchando en el puerto 22. Dentro del contenedor aparece `0.0.0.0`, pero la publicación de Docker en el equipo anfitrión continúa limitada a `127.0.0.1:22222`.

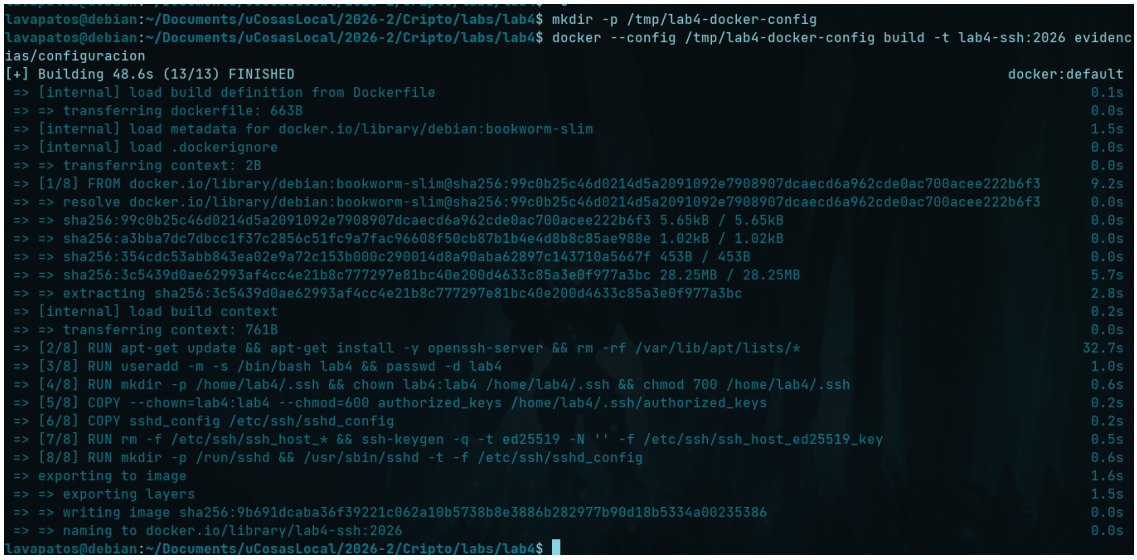


Figura 3: Construcción correcta de la imagen `lab4-ssh:2026`.

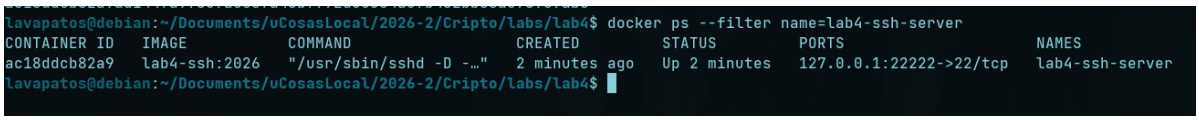


Figura 4: Contenedor activo y puerto SSH publicado únicamente en loopback.

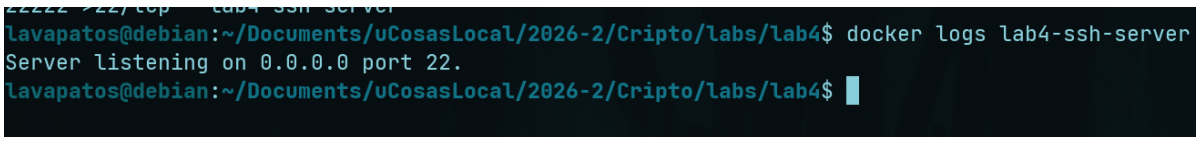


Figura 5: Servidor SSH escuchando correctamente dentro del contenedor.

3.1.2. Configuración del servidor

En `sshd_config` fijé de forma explícita los cuatro algoritmos pedidos. La configuración usa `curve25519-sha256` para el intercambio de claves, `aes256-ctr` para el cifrado, `hmac-sha2-256` para la integridad y `ssh-ed25519` como algoritmo de clave de host.

También deshabilité el inicio de sesión por contraseña y permití solamente autenticación por clave pública para el usuario `lab4`. El acceso de `root` y los reenvíos de X11, agente, puertos y túneles quedaron desactivados. La configuración completa utilizada se muestra en la Figura 6.

```
lavapatos@debian:~/Documents/uCosasLocal/2026-2/Cripto/labs/lab4$ touch evidencias/configuracion/sshd_config
lavapatos@debian:~/Documents/uCosasLocal/2026-2/Cripto/labs/lab4$ sudo nano evidencias/configuracion/sshd_config
[sudo] password for lavapatos:
lavapatos@debian:~/Documents/uCosasLocal/2026-2/Cripto/labs/lab4$ cat evidencias/configuracion/sshd_config
Port 22
AddressFamily inet
ListenAddress 0.0.0.0

HostKey /etc/ssh/ssh_host_ed25519_key
HostKeyAlgorithms ssh-ed25519
KexAlgorithms curve25519-sha256
Ciphers aes256-ctr
MACs hmac-sha2-256

PubkeyAuthentication yes
AuthenticationMethods publickey
AuthorizedKeysFile .ssh/authorized_keys
PasswordAuthentication no
KbdInteractiveAuthentication no
PermitEmptyPasswords no
PermitRootLogin no
UsePAM no
AllowUsers lab4

X11Forwarding no
AllowAgentForwarding no
AllowTcpForwarding no
GatewayPorts no
PermitTunnel no
UseDNS no
PrintMotd no
LogLevel VERBOSE

Subsystem sftp internal-sftp
lavapatos@debian:~/Documents/uCosasLocal/2026-2/Cripto/labs/lab4$
```

Figura 6: Configuración aplicada al servidor SSH.

Para comprobar que el servidor cargó el archivo y no solamente que este contenía los valores correctos, consulté su configuración efectiva con `sshd -T`. La salida confirmó exactamente `curve25519-sha256`, `aes256-ctr`, `hmac-sha2-256` y `ssh-ed25519`.

```
lavapatos@debian:~/Documents/uCosasLocal/2026-2/Cripto/labs/lab4$ docker exec lab4-ssh-server sshd -T | grep -E '^(kexalgorithms|ciphers
|macs|hostkeyalgorithms) '
ciphers aes256-ctr
macs hmac-sha2-256
kexalgorithms curve25519-sha256
hostkeyalgorithms ssh-ed25519
lavapatos@debian:~/Documents/uCosasLocal/2026-2/Cripto/labs/lab4$
```

Figura 7: Algoritmos efectivos informados por `sshd -T`.

3.1.3. Explicación de los algoritmos utilizados

Los algoritmos cumplen tareas distintas dentro de SSH. No son cuatro alternativas para hacer lo mismo, sino que se usan juntos durante el establecimiento y la comunicación de la sesión.

`curve25519-sha256.` Lo usé para el intercambio de claves. El cliente y el servidor usan X25519 para calcular un secreto compartido sin enviarlo directamente por la red. SHA-256 se usa en el hash del intercambio, que después participa en la generación de las claves de sesión. Este algoritmo no cifra por sí solo el tráfico y tampoco autentica al servidor. Su uso en SSH está definido en la sección 3 de la RFC 8731.

`aes256-ctr.` Es el algoritmo que cifra el tráfico SSH después del intercambio de claves. Usa AES con una clave de 256 bits y trabaja en modo contador. Su función es dar confidencialidad al contenido de los paquetes. No comprueba si un paquete fue modificado, porque esa tarea corresponde al HMAC. Los modos AES en contador para SSH están definidos en la sección 4 de la RFC 4344.

`hmac-sha2-256.` Comprueba la integridad de cada paquete mediante HMAC con SHA-256. El cálculo incluye el número de secuencia y el paquete SSH sin cifrar. Si alguien cambia el contenido durante el envío, la verificación ya no coincide y el receptor puede detectar la alteración. Este algoritmo no cifra el tráfico. `hmac-sha2-256` está especificado en la sección 2 de la RFC 6668, sobre el mecanismo de integridad descrito por la RFC 4253.

`ssh-ed25519.` Lo usé como algoritmo de firma para autenticar al servidor. El servidor firma el hash del intercambio con su clave privada Ed25519 y el cliente verifica la firma con la clave pública. Así se comprueba que el servidor posee la clave privada asociada a la huella presentada durante la conexión. Ed25519 firma, pero no cifra. En este laboratorio también usé una clave Ed25519 para autenticar al usuario `lab4`. Su uso como algoritmo de clave pública de SSH está definido en la RFC 8709.

En conjunto, Curve25519 establece el secreto compartido, Ed25519 autentica el intercambio, AES-256-CTR cifra los paquetes y HMAC-SHA-256 comprueba su integridad.

3.1.4. Conexión cliente-servidor

Me conecté desde el equipo anfitrión al servidor publicado en `127.0.0.1:22222`. Para autenticar al usuario utilicé la clave privada Ed25519 creada exclusivamente para este laboratorio. En la primera conexión el cliente mostró la huella de la clave de host Ed25519 y la acepté para guardarla en `known_hosts`.

La conexión no solicitó contraseña y abrió el prompt `lab4@ac18ddcb82a9`. Finalmente ejecuté `whoami` dentro del servidor y la respuesta fue `lab4`, lo que confirma que la sesión se estableció dentro del contenedor con el usuario esperado.

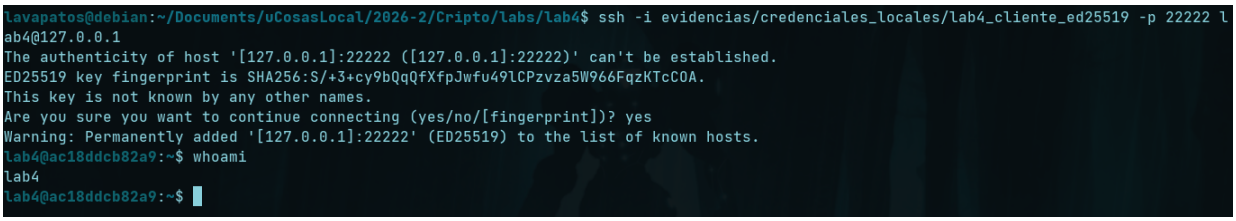


Figura 8: Conexión SSH funcional y comprobación del usuario remoto.

3.1.5. Análisis de tráfico

Capturé la conexión sobre la interfaz `lo` usando el filtro `tcp port 22222`. La captura guardada contiene 38 paquetes. Aunque el contenido de la sesión queda cifrado después del intercambio de claves, antes de ese punto todavía es posible observar información necesaria para establecer la conexión.

1. Establecimiento de la conexión TCP.

Las tramas 1, 2 y 3 muestran el intercambio `SYN`, `SYN, ACK` y `ACK`. También se observan el puerto efímero `52804` del cliente y el puerto `22222` del servidor. Ambas direcciones IP son `127.0.0.1` porque la prueba se realizó completamente sobre loopback. Estos datos permiten reconocer los extremos y confirmar que la conexión TCP se estableció correctamente.

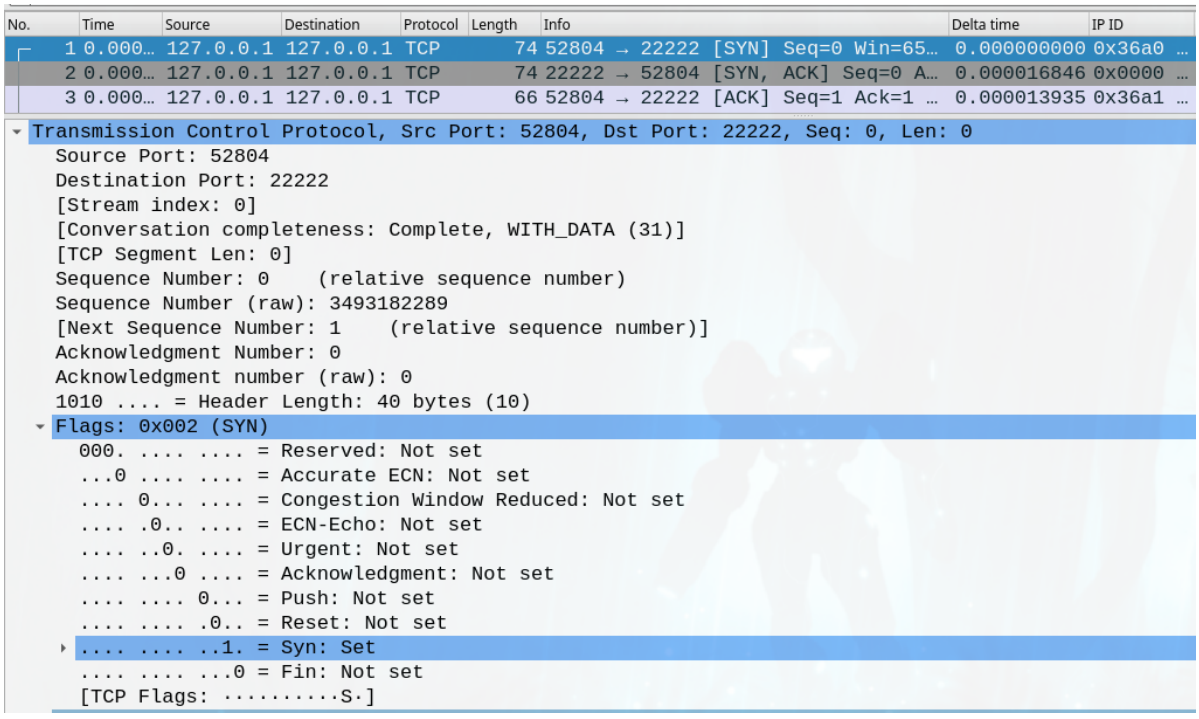


Figura 9: Handshake TCP de tres pasos antes de iniciar SSH.

2. Versiones del protocolo SSH.

Antes de activar el cifrado, cliente y servidor intercambian su cadena de identificación. En la trama 4 se observa `SSH-2.0-OpenSSH_9.2p1 Debian-2+deb12u10` y la dirección `client-to-server`. Esta información revela la versión del protocolo y la implementación utilizada, pero no contiene la clave privada ni las credenciales del usuario.

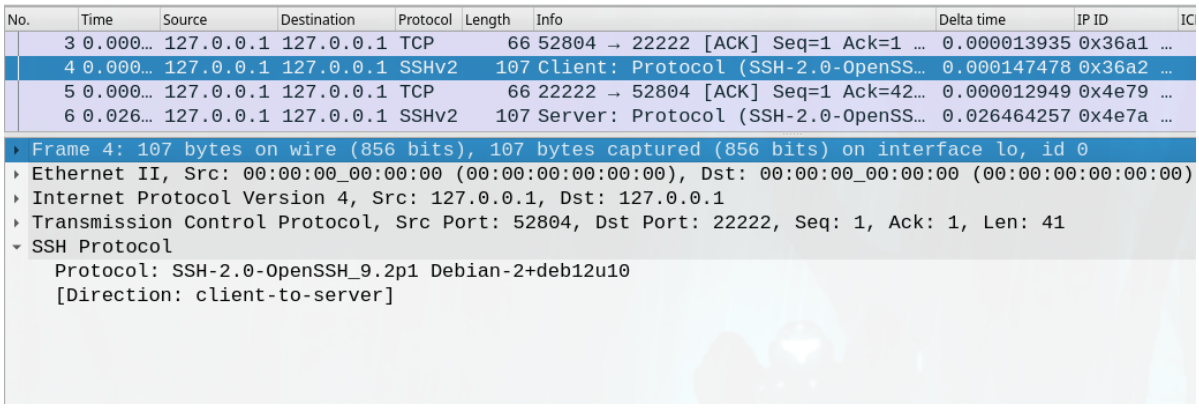


Figura 10: Cadena de versión SSH enviada antes de activar el cifrado.

3. Algoritmos ofrecidos en KEXINIT.

La trama 9 corresponde al mensaje KEXINIT del servidor. En ella aparecen las cuatro propuestas configuradas:

- Intercambio de claves: `curve25519-sha256`.
- Clave de host: `ssh-ed25519`.
- Cifrado: `aes256-ctr`.
- Integridad: `hmac-sha2-256`.

AES y HMAC se anuncian por separado para las direcciones cliente-servidor y servidor-cliente. También aparece una extensión de negociación estricta de OpenSSH:

`kex-strict-s-v00@openssh.com`

Esta extensión no es un segundo algoritmo KEX configurado por mí.

Por lo tanto, un observador puede conocer direcciones, puertos, tiempos, tamaños, versiones y algoritmos negociados. Después del mensaje `New Keys`, los comandos, las credenciales y el contenido transportado por la sesión ya no aparecen en texto plano.

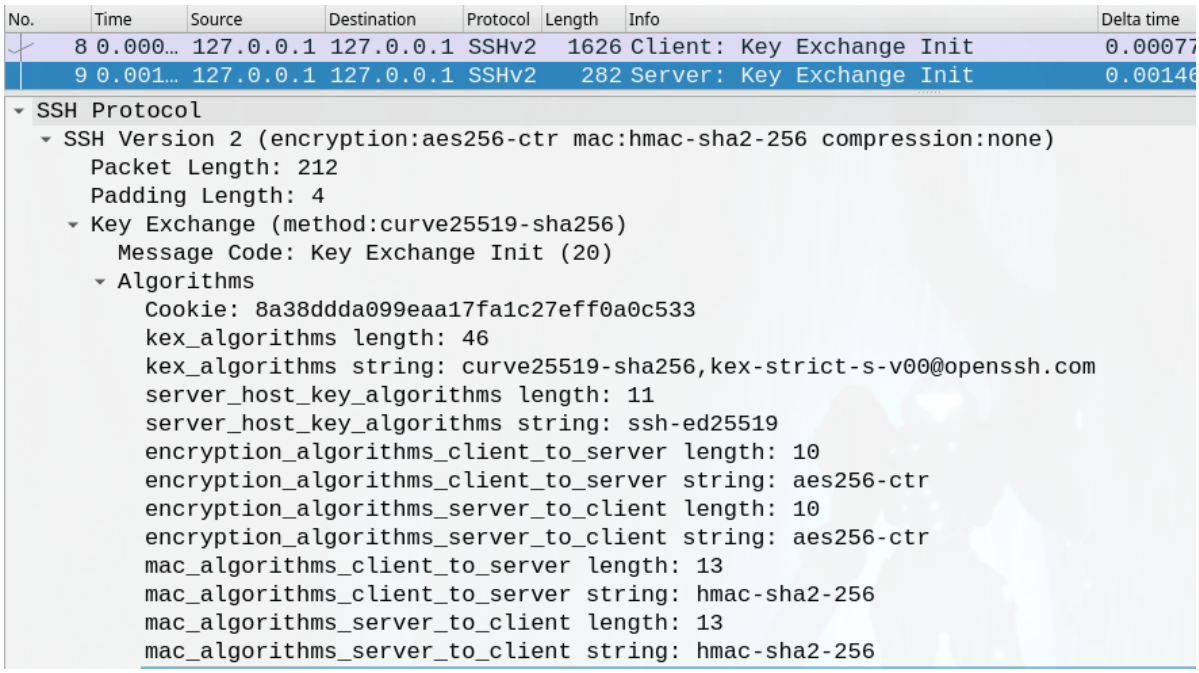


Figura 11: KEXINIT del servidor con los cuatro algoritmos configurados.

3.2. Actividad 2

3.2.1. Identificación del paquete RSA_PIN

La captura oficial contiene 41 paquetes TCP enviados desde `10.10.10.10:49152` hacia `10.10.10.20:4444`. Para localizar el paquete que transporta el PIN usé el filtro de Wireshark `tcp.payload contains "RSA_PIN"`.

El filtro encontró solamente la trama 1. Su payload tiene 280 bytes y comienza con la etiqueta ASCII `RSA_PIN|`. Después de los 8 bytes de la etiqueta aparecen los 16 bytes de salt y los 256 bytes del ciphertext RSA. En esta etapa se identifica la estructura, pero el PIN todavía no es visible en texto plano.

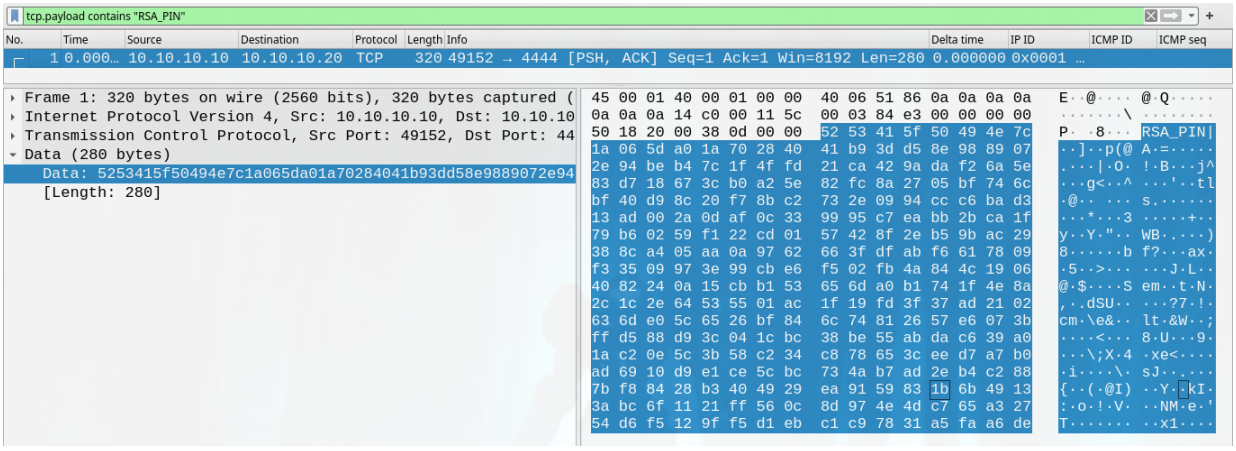


Figura 12: Trama 1 identificada mediante la etiqueta `RSA_PIN|`.

3.2.2. Identificación de los paquetes AESGCM

Para separar las transacciones usé el filtro `tcp.payload contains "AESGCM"`. Wireshark mostró 40 coincidencias, que corresponden a las tramas 2 a 41. Todas comienzan con la etiqueta ASCII `AESGCM|`.

Cada payload contiene, después de la etiqueta, un nonce de 12 bytes, el ciphertext de largo variable y un tag de autenticación de 16 bytes. La Figura 13 muestra el primer paquete seleccionado y el contador `Displayed: 40` en la barra inferior.

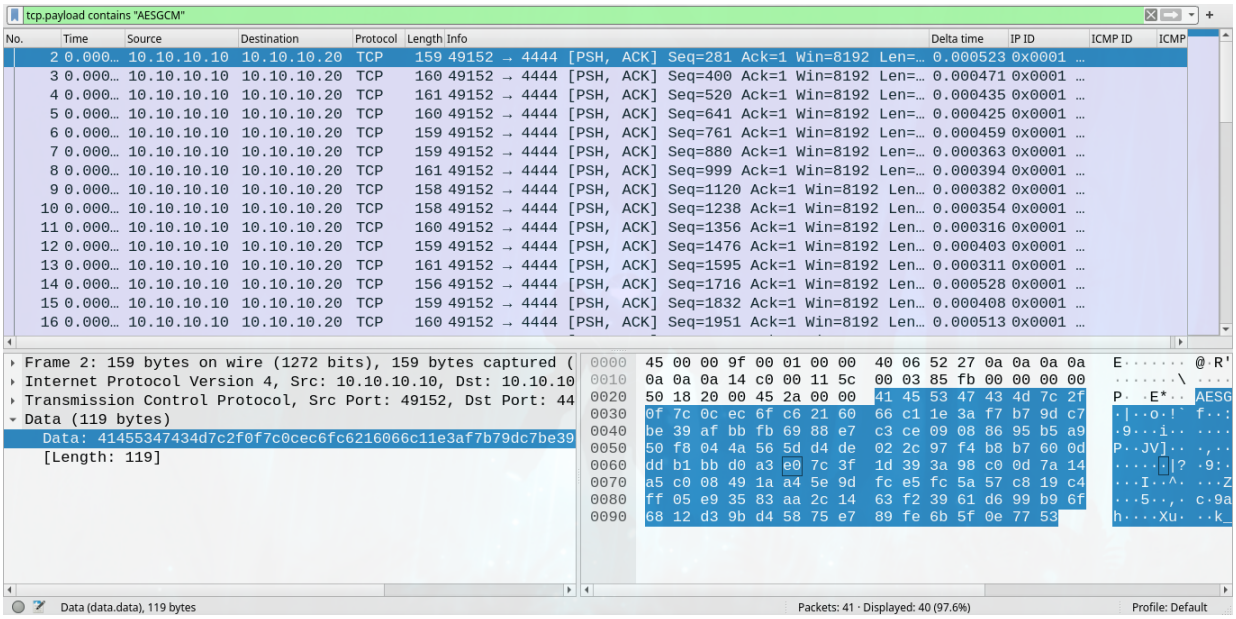


Figura 13: Cuarenta paquetes identificados mediante la etiqueta `AESGCM|`.

3.2.3. Recuperación del PIN y derivación de la clave AES

Leí el payload TCP de la trama 1 y separé sus campos usando posiciones fijas. Los primeros 8 bytes corresponden a `RSA_PIN|`; los bytes 8 a 23 contienen la salt de 16 bytes y, desde el byte 24 hasta el final, se encuentra el ciphertext RSA de 256 bytes.

La salt extraída fue:

`1a065da01a70284041b93dd58e988907`

El ciphertext comienza con `2e94beb47c1f4ffd21ca429adaf26a5e`. Lo descifré con la clave privada RSA entregada, usando OAEP con SHA-256 y MGF1-SHA-256. El resultado fue el PIN `4827`.

La salt no es una clave secreta. Su función es hacer que la derivación produzca un resultado distinto aunque se repita el mismo PIN y dificultar el uso de tablas precalculadas. En este caso es especialmente importante porque un PIN de cuatro dígitos tiene pocas combinaciones posibles.

Con el PIN recuperado como entrada y la salt anterior derivé 32 bytes mediante Scrypt. Usé exactamente $n = 2^{14}$, $r = 8$ y $p = 1$. El parámetro n determina el costo de cómputo y memoria de la derivación, r controla el tamaño de los bloques internos y p indica el grado de paralelización. Scrypt es determinista, por lo que cambiar la salt o cualquiera de esos valores produciría otra clave y el descifrado AES-GCM fallaría.

La clave AES-256 obtenida fue:

`f64d739ad8bd559fcd4bde7b9aecb83ff28d200331da635d7409b7133b16eab7`

```
lavapatos@debian:~/Documents/uCosasLocal/2026-2/Cripto/labs/lab4$ python3 evidencias/scripts/descifrar_transacciones.py
Trama RSA_PIN: 1
Salt (16 bytes): 1a065da01a70284041b93dd58e988907
Ciphertext RSA extraído: 256 bytes
Primeros 16 bytes: 2e94beb47c1f4ffd21ca429adaf26a5e
PIN recuperado con OAEP SHA-256: 4827
Scrypt: n=2**14, r=8, p=1, clave de 32 bytes
Clave AES-256: f64d739ad8bd559fcd4bde7b9aecb83ff28d200331da635d7409b7133b16eab7
Paquetes AES-GCM: 40

Transacciones recuperadas:
Nonce de ejemplo (12 bytes): 2f0f7c0cec6fc6216066c11e
Ciphertext de ejemplo: 84 bytes
Tag de ejemplo (16 bytes): 6f6812d39bd45875e789fe6b5f0e7753
```

Figura 14: Recuperación del PIN y derivación de la clave AES-256 mediante Scrypt.

3.2.4. Descifrar los mensajes

Para cada payload con la etiqueta `AESGCM|` descarté primero sus 7 bytes. Después separé los siguientes 12 bytes como nonce y los últimos 16 como tag; los bytes intermedios corresponden al ciphertext. En la trama 2 obtuve el nonce `2f0f7c0cec6fc6216066c11e`, 84 bytes de ciphertext y el tag `6f6812d39bd45875e789fe6b5f0e7753`.

Usé la clave AES-256 derivada anteriormente para descifrar cada mensaje. La llamada `AESGCM.decrypt` recibe el ciphertext unido a su tag, por lo que además de recuperar el texto

comprueba su autenticidad. Si el nonce, la clave, el ciphertext o el tag fueran incorrectos, la biblioteca produciría una excepción `InvalidTag`. Los 40 paquetes fueron descifrados sin esa excepción, de modo que la verificación terminó en `40/40`.

Cada texto recuperado trae los campos `ID`, `DE`, `A` y `MONTO`. El script los presenta como identificador, remitente, destinatario y monto, respectivamente. Este fue el código utilizado:

Listing 1: Script utilizado para recuperar y presentar las transacciones.

```
from pathlib import Path

from scapy.all import TCP, rdpcap
from cryptography.exceptions import InvalidTag
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives.kdf.scrypt import Scrypt


pcap = Path("documentos_lab/captura_y_claves_originales/transacciones_cifradas.pcap")
clave_pem = Path("documentos_lab/captura_y_claves_originales/clave_privada.pem")

paquetes = rdpcap(str(pcap))
payloads = []

for numero, paquete in enumerate(paquetes, 1):
    if TCP in paquete:
        payloads.append((numero, bytes(paquete[TCP].payload)))

trama_rsa, datos_rsa = next(
    (numero, datos) for numero, datos in payloads if datos.startswith(b"RSA_PIN|")
)

paquetes_aes = [
    (numero, datos)
    for numero, datos in payloads
    if datos.startswith(b"AESGCM|")
]

salt = datos_rsa[8:24]
ciphertext_rsa = datos_rsa[24:]

clave_privada = serialization.load_pem_private_key(
    clave_pem.read_bytes(),
    password=None,
)

pin = clave_privada.decrypt(
    ciphertext_rsa,
    padding.OAEP(
        mgf=padding.MGF1(hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None,
    ),
)

clave_aes = Scrypt(
    salt=salt,
    length=32,
    n=2**14,
    r=8,
    p=1,
).derive(pin)

```

```
print(f"Trama RSA_PIN: {trama_rsa}")
print(f"Salt (16 bytes): {salt.hex()}")
print(f"Ciphertext RSA extraído: {len(ciphertext_rsa)} bytes")
print(f"Primeros 16 bytes: {ciphertext_rsa[:16].hex()}")
print(f"PIN recuperado con OAEP SHA-256: {pin.decode()}")
print("Scrypt: n=2**14, r=8, p=1, clave de 32 bytes")
print(f"Clave AES-256: {clave_aes.hex()}")
print(f"Paquetes AES-GCM: {len(paquetes_aes)}")

aesgcm = AESGCM(clave_aes)
autenticadas = 0

print("\nTransacciones recuperadas:")

for indice, (trama, datos) in enumerate(paquetes_aes, 1):
    cuerpo = datos[7:]
    nonce = cuerpo[:12]
    cifrado_y_tag = cuerpo[12:]
    ciphertext = cifrado_y_tag[:-16]
    tag = cifrado_y_tag[-16:]

    try:
        texto = aesgcm.decrypt(nonce, cifrado_y_tag, None).decode("utf-8")
    except InvalidTag:
        print(f"Trama {trama}: autenticidad inválida")
        continue

    campos = {}
    for parte in texto.split("|")[1:]:
        nombre, valor = parte.split("=", 1)
        campos[nombre] = valor

    if indice == 1:
        print(f"Nonce de ejemplo (12 bytes): {nonce.hex()}")
        print(f"Ciphertext de ejemplo: {len(ciphertext)} bytes")
        print(f"Tag de ejemplo (16 bytes): {tag.hex()}\n")

    print(
        f"{indice:02d}. ID: {campos['ID']} | "
        f"Remitente: {campos['DE']} | "
        f"Destinatario: {campos['A']} | "
        f"Monto: {campos['MONTO']}"
    )
    autenticadas += 1

print(f"\nAutenticidad AES-GCM verificada: {autenticadas}/{len(paquetes_aes)}")
```

```
01. ID: TRX-426477 | Remitente: Diego Ramirez | Destinatario: Constanza Vidal | Monto: 763923
02. ID: TRX-346837 | Remitente: Catalina Vargas | Destinatario: Valentina Rojas | Monto: 58911
03. ID: TRX-412154 | Remitente: Sebastian Munoz | Destinatario: Constanza Vidal | Monto: 485213
04. ID: TRX-071484 | Remitente: Felipe Morales | Destinatario: Tomas Sepulveda | Monto: 314718
05. ID: TRX-805221 | Remitente: Isidora Silva | Destinatario: Javiera Fuentes | Monto: 104470
06. ID: TRX-971697 | Remitente: Valentina Rojas | Destinatario: Camila Torres | Monto: 785714
07. ID: TRX-139273 | Remitente: Catalina Vargas | Destinatario: Nicolas Herrera | Monto: 913757
08. ID: TRX-652114 | Remitente: Camila Torres | Destinatario: Fernanda Araya | Monto: 855548
09. ID: TRX-432361 | Remitente: Vicente Soto | Destinatario: Matias Gonzalez | Monto: 818686
10. ID: TRX-818929 | Remitente: Antonia Castro | Destinatario: Tomas Sepulveda | Monto: 885868
11. ID: TRX-871181 | Remitente: Catalina Vargas | Destinatario: Camila Torres | Monto: 514444
12. ID: TRX-157566 | Remitente: Tomas Sepulveda | Destinatario: Javiera Fuentes | Monto: 436301
13. ID: TRX-186243 | Remitente: Vicente Soto | Destinatario: Daniela Reyes | Monto: 698110
14. ID: TRX-121024 | Remitente: Isidora Silva | Destinatario: Sebastian Munoz | Monto: 623481
15. ID: TRX-339097 | Remitente: Matias Gonzalez | Destinatario: Constanza Vidal | Monto: 77226
16. ID: TRX-663549 | Remitente: Isidora Silva | Destinatario: Antonia Castro | Monto: 576901
17. ID: TRX-367001 | Remitente: Fernanda Araya | Destinatario: Tomas Sepulveda | Monto: 282567
18. ID: TRX-188807 | Remitente: Camila Torres | Destinatario: Antonia Castro | Monto: 195173
19. ID: TRX-096936 | Remitente: Felipe Morales | Destinatario: Antonia Castro | Monto: 359917
20. ID: TRX-695623 | Remitente: Catalina Vargas | Destinatario: Nicolas Herrera | Monto: 619299
21. ID: TRX-407749 | Remitente: Camila Torres | Destinatario: Sebastian Munoz | Monto: 182369
22. ID: TRX-069658 | Remitente: Valentina Rojas | Destinatario: Javiera Fuentes | Monto: 556520
23. ID: TRX-232469 | Remitente: Felipe Morales | Destinatario: Tomas Sepulveda | Monto: 23769
24. ID: TRX-940466 | Remitente: Vicente Soto | Destinatario: Fernanda Araya | Monto: 292293
25. ID: TRX-755002 | Remitente: Matias Gonzalez | Destinatario: Daniela Reyes | Monto: 946989
26. ID: TRX-962321 | Remitente: Felipe Morales | Destinatario: Daniela Reyes | Monto: 589780
27. ID: TRX-324981 | Remitente: Diego Ramirez | Destinatario: Daniela Reyes | Monto: 971045
28. ID: TRX-337011 | Remitente: Antonia Castro | Destinatario: Felipe Morales | Monto: 777198
29. ID: TRX-995006 | Remitente: Vicente Soto | Destinatario: Sebastian Munoz | Monto: 587590
30. ID: TRX-630572 | Remitente: Camila Torres | Destinatario: Sebastian Munoz | Monto: 332702
31. ID: TRX-517969 | Remitente: Catalina Vargas | Destinatario: Daniela Reyes | Monto: 484770
32. ID: TRX-447236 | Remitente: Matias Gonzalez | Destinatario: Constanza Vidal | Monto: 331352
33. ID: TRX-852645 | Remitente: Constanza Vidal | Destinatario: Felipe Morales | Monto: 343065
34. ID: TRX-188241 | Remitente: Camila Torres | Destinatario: Constanza Vidal | Monto: 781107
35. ID: TRX-713923 | Remitente: Diego Ramirez | Destinatario: Catalina Vargas | Monto: 21358
36. ID: TRX-362125 | Remitente: Camila Torres | Destinatario: Sebastian Munoz | Monto: 154118
37. ID: TRX-961549 | Remitente: Felipe Morales | Destinatario: Sebastian Munoz | Monto: 614169
38. ID: TRX-965060 | Remitente: Tomas Sepulveda | Destinatario: Antonia Castro | Monto: 778912
39. ID: TRX-437498 | Remitente: Valentina Rojas | Destinatario: Matias Gonzalez | Monto: 440902
40. ID: TRX-367112 | Remitente: Matias Gonzalez | Destinatario: Isidora Silva | Monto: 91711
```

Figura 15: Cuarenta transacciones recuperadas con ID, remitente, destinatario y monto.

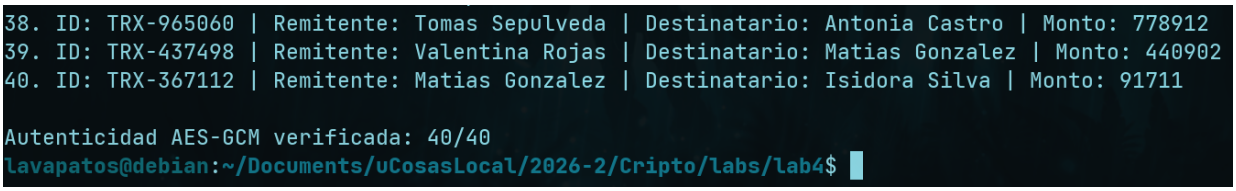


Figura 16: Verificación de autenticidad AES-GCM completada para los 40 mensajes.

3.2.5. RSA frente a criptografía post-cuántica

Mi conclusión es que las claves RSA entregadas no permitirían descifrar mensajes protegidos exclusivamente mediante un algoritmo post-cuántico como ML-KEM. En este laboratorio la clave privada pudo recuperar el PIN porque el ciphertext había sido generado con la clave pública RSA correspondiente y se usó OAEP con SHA-256. Es decir, tanto el formato de la clave como el del ciphertext pertenecían a RSA.

ML-KEM no es RSA con otro relleno. Este mecanismo genera una clave de encapsulación y otra de decapsulación propias. Para recuperar el secreto compartido se necesita la clave de decapsulación ML-KEM que corresponde a ese ciphertext. Una clave privada RSA no tiene el formato ni la estructura interna requerida por ML-KEM, por lo que no puede reemplazarla.

El algoritmo de Shor explica por qué RSA sería vulnerable frente a un computador cuántico suficientemente potente. Sin embargo, romper RSA no convierte una clave RSA en una clave post-cuántica ni permite usarla para decapsular ML-KEM.

El único caso que requiere un matiz es un esquema híbrido. Si un protocolo combinara RSA y ML-KEM, conocer la clave RSA podría comprometer el componente RSA, pero no entregaría automáticamente el secreto del componente post-cuántico. En una construcción bien diseñada, la clave final depende de ambos resultados, de modo que romper solamente RSA no debería bastar mientras ML-KEM continúe siendo seguro. El resultado exacto dependería del método usado por el protocolo para combinar ambos secretos.

Conclusiones y comentarios

El servidor SSH quedó funcionando dentro del contenedor y accesible solamente desde `127.0.0.1:22222`. La configuración efectiva confirmó `curve25519-sha256`, `aes256-ctr`, `hmac-sha2-256` y `ssh-ed25519`. Al principio Wireshark mostraba el tráfico del puerto 22222 solamente como TCP y fue necesario usar *Decode As* para interpretarlo como SSH. Esta fue la parte que más me costó entender. Una vez configurado pude observar las direcciones, los puertos, las versiones de OpenSSH y los algoritmos negociados. Aunque SSH cifra los comandos y sus respuestas, esa información todavía queda visible en la captura.

En la captura entregada identifiqué una trama `RSA_PIN|` y 40 tramas `AESGCM|`. Del primer payload extraje la salt de 16 bytes y el ciphertext RSA de 256 bytes. El descifrado OAEP con SHA-256 recuperó el PIN `4827`; después, Scrypt con $n = 2^{14}$, $r = 8$ y $p = 1$ produjo

la clave AES-256. Con esa clave separé el nonce, el ciphertext y el tag de cada mensaje. Esto formó una cadena en la que cada resultado era necesario para el paso siguiente: RSA permitió recuperar el PIN, Scrypt derivó la clave y AES-GCM descifró las transacciones. La verificación `40/40` no solo indica que los mensajes pudieron leerse, sino también que sus tags fueron válidos en los 40 casos.

También quedó claro que una clave privada no sirve para cualquier esquema de cifrado. La clave RSA entregada permite descifrar el ciphertext RSA correspondiente, pero no puede reemplazar una clave de decapsulación ML-KEM. Incluso en un sistema híbrido, comprometer RSA no debería bastar por sí solo si el componente post-cuántico y el método de combinación siguen siendo seguros.

En general, la actividad se sintió muy guiada. Docker permitió aislar el servidor y probar la configuración sin modificar directamente el equipo anfitrión, pero no lo consideré esencial para el objetivo porque SSH también podía configurarse de forma directa. Para mí fue una herramienta más dentro del procedimiento. También habría preferido una estructura menos rígida para el informe y mayor libertad para ajustar la geometría de la plantilla, ya que con las capturas de pantalla fue difícil evitar imágenes demasiado grandes o espacios mal aprovechados.

4. Bibliografia

- Captura y Claves
- Repositorio del Laboratorio 4 en GitHub.
- IETF, RFC 8731: intercambio de claves Curve25519 en SSH, sección 3.
- IETF, RFC 4344: modos AES-CTR para SSH, sección 4.
- IETF, RFC 6668: HMAC-SHA2 para SSH, sección 2.
- IETF, RFC 8709: Ed25519 para SSH.
- IETF, RFC 4253: integridad de paquetes en el protocolo de transporte SSH, sección 6.4.
- IETF, RFC 8017: RSAES-OAEP, sección 7.1.
- NIST, FIPS 203: estándar ML-KEM.
- NIST, SP 800-227: mecanismos de encapsulación y esquemas híbridos.
- NIST, IR 8547: transición hacia criptografía post-cuántica.